

Japi Base Editor – User Manual

*This is the reference for **JBE**, the full-screen text editor of the Japi Base Computer. It is where you read, write and run your programs. It starts with a tour of the screen, then how to move around and select text, then how the menus work, after which each menu is described in detail.*

How to read the keys

Notation	Meaning
Alt+F	hold Alt and press F
Ctrl+S	hold Ctrl and press S
Shift+→	hold Shift and press the right-arrow key
↑ ↓ ← →	the four arrow keys
Enter, Esc, Tab	those named keys

A tour of the interface

When the editor is running the screen is divided into a few fixed regions. Knowing them makes the rest of the manual easier to follow.

The text area

The large middle area is where your file is shown and edited. The **cursor** – the block that marks where your next keystroke will land – sits somewhere in it. The text scrolls under the cursor so it always stays in view.

A line that is too long to fit across the screen is **wrapped** onto the next row, with a small red continuation marker at the right edge so you can see that the row below still belongs to the same line.

The menu bar

The top row is the **menu bar**. It is always visible and lists the seven menus:

> **File Edit View Search Macro Options Run**

Everything the editor can do is reachable from these menus, so the bar doubles as a complete list of its features. How to open and use them is described under *Using the menus* below.

The status bar

The bottom row is the **status bar**. While you are editing it shows, from left to right:

- **Ln** and **Col** – the line and column the cursor is on (both count from 1);
- the number of lines in the file;
- the **file name**, with a ******* after it when the file has unsaved changes;
- **[REC]** while a macro is recording.

The status bar doubles as the editor's prompt line: when it needs an answer from you – Save As, Find, Replace or "discard unsaved changes?" – it turns **black-on-yellow** and asks you right there, then returns to the normal readout.

The scroll bar

The right-hand column of the text area is a **scroll bar**, in the QuickBASIC style. A small **^** at the top or **v** at the bottom appears when there is more text to scroll to in that direction. A pale block – the *thumb* – shows roughly where you are in the file and how much of it fits on screen. It is an indicator you read, not something you click: you move through the file with the keyboard, as the next section explains.

Moving around in your text

You move the cursor with the keys below. None of these change your text; they only move the cursor. The view scrolls by itself to keep the cursor in sight, so you rarely have to think about scrolling at all.

Key	Moves the cursor
← / →	one character left or right
↑ / ↓	one row up or down, keeping roughly the same column
Home	to the start of the line
End	to the end of the line
PgUp	up by one full screen
PgDn	down by one full screen

Changing the text

Three keys edit at the cursor:

- **Backspace** deletes the character to the **left** of the cursor.
- **Delete** deletes the character to the **right** of the cursor.
- **Enter** breaks the line in two at the cursor. The new line ****keeps the**

indentation** of the line you were on, so code stays lined up as you type.

The editor is always in **insert** mode: whatever you type is pushed in at the cursor and the rest of the line slides to the right. There is no overwrite mode, so you never type over text by accident – to replace text you select it first (next section) and then type. The **Insert** key, which on other editors flips between insert and overwrite, has no role here.

Selecting text

Many commands act on a **selection** – a stretch of text you have marked. The editor shows the selected text highlighted. You make a selection by moving the cursor while you hold **Shift**. There are two kinds.

A normal selection – Shift + movement

Hold **Shift** and use any of the movement keys – the arrows, **Home**, **End**, **PgUp**, **PgDn** – to sweep the highlight from where the cursor started to where it is now. This is the everyday kind: it follows the text, wrapping from the end of one line to the start of the next, so it can cover whole lines and paragraphs. It is exactly how selecting works in any modern editor.

The mark is dropped at the cursor the moment you first press a Shift-movement. From there it grows or shrinks as you keep moving with Shift held.

A rectangular selection – Ctrl+Shift + movement

Holding **Ctrl+Shift** while you move makes the highlight become a **rectangle** – a column block. Rather than following the text from line to line, it marks the same range of columns on every row between the start and the cursor. This is ideal for editing things that line up in columns, such as the start of a run of lines or a table laid out with spaces.

A rectangular block keeps its shape when you copy and paste it: it drops back in as a rectangle at the cursor, not as one long run of text.

What a selection responds to

Once text is selected:

- **Typing** any character **replaces** the whole selection with what you type.
- **Backspace** or **Delete** removes the selection.
- **Cut**, **Copy** and **Paste** act on it (see the Edit menu below).
- Any plain movement **without Shift** clears the selection and leaves the text

untouched – so a stray arrow key simply deselects, it never deletes.

Indenting a block – Tab and Shift+Tab

With a selection active, **Tab** shifts every line it touches one tab width to the **right**. **Shift+Tab** shifts them back to the **left**. This indents or out-dents a whole block in one keystroke. (With no selection, **Tab** just inserts spaces up to the next tab stop.)

Tab stops sit every **2 columns**. The editor always inserts real spaces – never an actual tab character – so your file lines up the same way everywhere.

Matching brackets

When the cursor sits **on a bracket** – **() [] { }** – or just to the right of one, the editor finds its partner and gives it a **grey highlight**, so you can see at a glance where a pair opens and closes (it looks inside nested pairs correctly and ignores brackets inside **"..."** strings). If the bracket has **no partner**, it turns **red**, which is a quick way to spot a missing or extra bracket.

Using the menus

The menu bar at the top is your map of the editor: open a menu and you can see and reach everything it does, so you never *have* to remember a shortcut.

Opening a menu

The first letter of each menu name is shown in **red**. To open a menu, hold **Alt** and press that letter:

Menu	Opens with
File	Alt+F
Edit	Alt+E
View	Alt+V
Search	Alt+S
Macro	Alt+M
Options	Alt+O
Run	Alt+R

A dropdown list folds down under the menu name.

Choosing an item – the red letter

Inside an open menu, **every item has one red letter** as well. To run an item you press **Alt + that letter**. The simplest way is to keep **Alt** held down the whole time: open the menu with **Alt+F**, then – still holding **Alt** – press **N** for *New*. So *File* → *New* is **Alt+F** followed by **Alt+N**.

Choosing an item – the arrow keys

You can also steer through the menus by hand:

- **← / →** move to the menu on the left or right. The bar wraps around, so going left from *File* lands on *Run*.
- **↑ / ↓** move the highlight up or down through the items of the open menu.

The list wraps top-to-bottom and **separator lines are skipped** – the highlight always lands on a real, selectable item.

- **Enter** runs the highlighted item.
- **Esc** closes the menu and does nothing.

Shortcuts – skipping the menu

Many items show a **shortcut** on their right-hand side, for example **Ctrl+S** next to *Save*. Those key combinations work **straight from the editor**, without opening any menu – they are the fast path for the things you do often. The menus and the shortcuts always do exactly the same thing; the shortcut is just a quicker way in.

Separator lines

Some dropdowns contain a thin horizontal line. It is only a **visual separator** that groups related items together (for instance, in the Edit menu it keeps *Cut / Copy*

/ Paste / Select All apart from Toggle Comment / Undo / Redo). You cannot select a separator; the arrow keys step over it.

The menus at a glance

Menu	What it is for
File	start, open, save and close your files
Edit	cut, copy and paste, comment or format, undo changes
View	split the screen into two panes and arrange them
Search	find text and find-and-replace
Macro	record a sequence of keystrokes and play it back
Options	syntax highlighting and the CPU speed
Run	run the BASIC program you are editing

Each menu gets its own section below.

File menu

The File menu handles your documents – starting a new one, opening an existing one, saving and closing. Open it with **Alt+F**.

> New · Open... · Save · Save As... · Close

Every item acts on the **active pane** (the one your cursor is in). When the screen is split in two, the other pane is left completely untouched – see the *View menu* for how the split works.

Whenever New, Open or Close would throw away unsaved changes, the editor first pops up a framed **Unsaved changes** box just above the status bar: press **Y** to discard them and go ahead, or any other key to cancel and keep your work.

New – Ctrl+N

Starts a fresh, empty document in the active pane, replacing whatever was there (if that document has unsaved changes, you are asked first). The new document is *untitled*: it has no file on disk yet, so the first time you Save it the editor asks for a name (see *Save As*).

Open... – Ctrl+O

Opens the **file picker** and loads the file you choose into the active pane. While the picker is open:

- the arrow keys move through the list and **Enter** opens the highlighted file;
- **Tab** switches drive between **A:** (the SD card) and **C:** (the built-in

storage), each time showing that drive's top folder;

- **Del** deletes the highlighted file (it asks you to confirm first);
- **Esc** closes the picker without opening anything.

The "... " after *Open* is the classic hint that the item leads to a dialog rather than acting at once. If the document already in the pane has unsaved changes,

opening another file asks you to confirm first.

Save – Ctrl+S

Writes the active document back to its file on disk. If the document is still untitled (it has no file yet), Save automatically turns into **Save As** so you can give it a name first – Save always does the sensible thing.

Save As...

Asks for a file name and saves the active document under it, adopting that name and location from then on. You type the name in a **framed box** at the bottom of the screen; press **Enter** to save or **Esc** to cancel. Use Save As to store an untitled document for the first time, or to keep a copy under a new name. (This item has no keyboard shortcut – reach it through the menu.)

Close – Ctrl+W

Empties the active pane back to a fresh untitled document. **The pane itself never disappears**; to go back to a single full-screen pane, turn the windowed view off again with *View → Toggle windowed view*. If the document has unsaved changes that would be lost, Close asks you to confirm first (the **Unsaved changes** box above). If the very same file is also shown in the other pane, nothing is lost, so Close just clears this view without asking.

No Exit

Unlike the old DOS editors, the File menu has **no Exit**. The Japi Base editor *is* the whole machine – there is nowhere to exit to, so it is simply always on.

Edit menu

The Edit menu holds the clipboard commands, Select All, the comment toggle, the formatter and undo / redo. Open it with **Alt+E**.

> Cut · Copy · Paste · Select All · — · Toggle Comment · Format · — · Undo · Redo

The first separator keeps the clipboard group apart from *Toggle Comment* and *Format*; the second sets off *Undo / Redo*. Cut, Copy and Paste all work on the **selection** described above, so make your selection first.

Cut – Ctrl+X

Copies the selection to the clipboard and then removes it from the text. With nothing selected, Cut does nothing.

Copy – Ctrl+C

Copies the selection to the clipboard and **leaves it in place**. The clipboard remembers whether the selection was a normal run or a rectangular block, so a later Paste puts it back in the same shape.

Paste – Ctrl+V

Inserts the clipboard at the cursor. A normal copy flows in as text at the cursor position; a rectangular copy drops back in as a rectangle, starting at the

cursor's column. Pasting over a selection replaces that selection first.

Select All – Ctrl+A

Selects the whole file in one keystroke – from the very first character to the last – as a normal (stream) selection. Handy before a Copy to grab everything, or before a Cut to clear the buffer. Any cursor movement without Shift clears the selection again.

Toggle Comment – Ctrl+G

Comments or un-comments lines by putting a BASIC comment marker – an apostrophe `'` – at the very **start** of the line. Run it on a single line (no selection needed) and it toggles that one line; run it on a selection and it toggles every line the selection touches. If the lines are already commented it removes the marker, otherwise it adds one – so the same key flips them either way. The marker always goes at the very beginning of the line, not in front of the first non-blank character.

Format

Format tidies up the whole BASIC program in the active pane in one step – or just the selected lines, if you have a selection. It does three things: it **re-indents** the block structure (two spaces per level, following IF, FOR, SUB, SELECT and the like), **trims the trailing spaces** off every line and **uppercases the BASIC keywords and built-in functions** to their standard spelling. Your variable names, your numbers, the text inside `"..."` strings and the text of comments are all left exactly as they were. It always formats as BASIC, whatever colour scheme the file happens to be using.

Format has no shortcut of its own – reach it through the menu (**Alt+E** then **Alt+F**). It is applied as two edits, so a full Undo of a format takes **two** presses of **Ctrl+Z**.

Undo – Ctrl+Z

Steps back through your edits, one change at a time. Typing is grouped into sensible chunks – roughly a word at a time, the way Word and other modern editors do it – so one Undo takes back a whole word rather than a single letter.

Redo – Ctrl+Y

Re-applies what you have just undone, as long as you have not typed anything new in between.

View menu

The View menu splits the screen into two panes and arranges them. Open it with **Alt+V**.

> Toggle windowed view · Other pane · Move divider up · Move divider down

The editor stacks the two panes one **above** the other, with a divider line between them. Only one pane is active at a time: the one your cursor is in, which is also the one the status bar reports on.

Toggle windowed view

This splits the screen into two panes and toggles back to one. Reach it with **Alt+V** then **Alt+W** – it has no shortcut of its own. The first time you turn it on, the screen divides in two and **both panes show the same file** – two views of one document, scrolled independently. This is handy for reading two distant parts of a long program at once, or for copying from one place to another. To give a pane its own file, just do **File → New** or **File → Open** in it; that detaches the pane onto its own document and leaves the other pane untouched.

Choosing it again turns the windowed view **off**: the screen returns to a single full-screen pane, keeping the pane you are in and dropping the other. If the pane being dropped has unsaved changes that would be lost, the editor asks you to confirm first.

Other pane – Ctrl+Tab

Moves the focus to the other pane, so your cursor and your typing go there instead. It does nothing when the screen is not split.

Move divider up / down – Ctrl+Up / Ctrl+Down

Shift the divider line one row up or down, giving one pane more room and the other less. The editor always keeps at least one line of text in each pane, so neither can be squeezed away completely. Both do nothing when the screen is not split.

Search menu

The Search menu finds text and does find-and-replace. Open it with **Alt+S**. Both items pop up a framed, **black-on-yellow box** just above the status bar, with a blinking caret where you type.

> Find... · Replace... · Go to Line...

Find... – Ctrl+F

Opens the Find box and starts an **incremental** search: as you type, the editor jumps to the first match from the cursor and **highlights** it. The box stays open so you can step through every match:

- **Enter**, **Ctrl+N** or **Ctrl+F** jump to the **next** match;
- **Ctrl+P** jumps to the **previous** match;
- both wrap around the ends of the document, so you can keep going in a circle;
- **Backspace** edits the search text, re-searching as you go;
- **Esc** closes the box, leaving the cursor on the last match.

If nothing matches, the box shows **<not found>**. The same key reminders are written in the box's title bar.

Replace... – Ctrl+R

Opens the Replace box, which works just like Find but with a second field: the search text on top and the replacement below it, after the word **with**. The box stays open until you press Esc:

- type the search text – the editor highlights matches as you go, exactly as Find does;

- **Tab** switches between the search field and the **with** field, so you can edit either;

- **Ctrl+N** and **Ctrl+P** step to the next and previous match (both wrap);
- **Enter** replaces the highlighted match with the **with** text and jumps to the next one, so you can tap Enter to roll through the matches you want;
- **Ctrl+A** replaces the current match and every one after it in one go;
- **Esc** closes the box.

The key reminders are written in the box's title bar. As with Find, the box shows **<not found>** when nothing matches.

Go to Line... – Ctrl+L

Opens a small box that asks for a **line number** and jumps the cursor straight there. Type the number (digits only), press **Enter** to jump or **Esc** to cancel. A number past the end of the file lands you on the last line.

On the full Japi Base Computer, you rarely need to type the number after an error: running a BASIC program that **stops on an error** drops the cursor on the offending line **automatically** when you return to the editor.

Macro menu

The Macro menu records a sequence of keystrokes once and plays it back, which is handy for a small edit you have to repeat many times. There is **one macro at a time** – exactly like the record-and-playback in Emacs, Notepad++ or Sublime Text. Open the menu with **Alt+M**.

> Record start/stop · Play

Record start/stop – Ctrl+T

Starts and stops **recording** your keystrokes. Press **Ctrl+T** to begin: the status bar shows **[REC]** while it records. Do the edit you want to capture, then press **Ctrl+T** again to stop. Starting a new recording replaces the previous one.

A macro holds up to **256 keystrokes**; if you go past that, recording simply stops there and keeps what it had so far.

Play – Ctrl+P

Plays the recorded keystrokes back at the cursor, as if you typed them again. The replay is not itself recorded, so it can never nest or run away. With no macro recorded yet, Play does nothing.

A typical run: **Ctrl+T**, make the edit on one line, **Ctrl+T** to stop, move to the next line, then tap **Ctrl+P** for each line you want the same edit on.

Options menu

The Options menu chooses the **syntax-highlighting** scheme for the file in the active pane and sets the **CPU speed**. Open it with **Alt+O**. Unlike the other menus its list is **rebuilt each time you open it**, because it reflects the schemes available right now.

> Syntax: None · Syntax: Z80 · Syntax: Basic · ... · CPU speed...

The list always begins with:

- **Syntax: None** – plain text, no colouring;
- the two **built-in** schemes, **Z80** assembly and **Basic**, which need no

files of their own;

and then it adds any **.syn** scheme files you have placed in **C:\config\syntax** on the built-in disk (see *Syntax files* below). Picking an item recolours the current file at once. Each item's red letter is the first letter of its name (**N** for None), so *Options* → *Basic* is **Alt+O** then **Alt+B**.

It is usually automatic

You rarely have to touch this menu: when you open a file the editor already picks a scheme from the file's **extension** – a **.bas** file gets Basic, a **.z80** file gets Z80 and so on. Options is there to **override** that, for example to turn colouring off or to pick a scheme of your own. The choice lasts while the file is open; reopening it detects from the extension again.

Syntax files

Your own colour schemes are plain **.syn** text files in **C:\config\syntax** on the built-in flash (drive C:). Each file names a scheme and lists its keywords and colours; the file name must match the scheme name in lower case (so a scheme called *MyBASIC* is the file **mybasic.syn**). Drop one in that folder and it shows up in this menu the next time you open it.

CPU speed

The last item, **CPU speed...**, opens a small dialog that sets how fast the processor runs. It offers the same three speeds as the Japi Base platform:

- **260 MHz** – the safe fallback;
- **324 MHz** – the default;
- **390 MHz** – an opt-in turbo.

Use **↑ / ↓** to choose, **Enter** to apply or **Esc** to cancel. The dialog marks the speed you are on now; applying a new one **reboots the machine** (about a second). A board that cannot hold the top speed steps itself back down on the next start, so 390 MHz is always safe to try.

Run menu

Open it with **Alt+R**. The Run menu holds the editor's **Japi Commander** – a small file manager for moving files between the drives.

> Japi Commander

Japi Commander – Ctrl+J

The Japi Commander is a two-pane file browser. Press **Ctrl+J** (or pick it from the menu) and it takes over the whole screen:

- The **left pane** shows the SD card (**A:**) and the **right pane** shows the built-in flash drive (**C:**). The pane with the **yellow frame** is the active one.

- **Tab** switches which pane is active.

- The **arrow keys**, **PgUp** / **PgDn** and **Home** / **End** move the highlight in the active pane; **Enter** opens a folder.

- **Ctrl+Tab** points the active pane at another drive.

- **Space** tags the highlighted file so an operation can act on several files at once; **Shift+Up** / **Shift+Down** extend the tagged run.

- **Ctrl+C** copies and **Ctrl+X** cuts the tagged (or highlighted) items –

whole folders included, with everything inside them; then switch to the other pane and **Ctrl+V** pastes them into the folder shown there. If a paste would overwrite a file of the same name, the Commander asks first: press **Y** to replace that one, **N** to skip it or **A** to replace all the rest without asking again.

- **Del** removes the tagged (or highlighted) items after a yellow confirmation line – press **Y** to go ahead. A folder is removed with everything inside it, so deleting a folder takes its whole contents along.

- **Ctrl+R** renames the highlighted file; **Ctrl+N** makes a new folder in the active pane.

- **Esc** (or **Ctrl+J** again) closes the Commander and returns you to the editor.

Its first job is the everyday one: copy a program or a configuration file from the SD card onto the built-in C: drive, or back. The copy is byte-for-byte, so binary files such as keyboard layouts come across intact.

Together these turn the Commander into a working file manager: copy or move files and whole folders between the two drives, delete what you no longer need, rename a file or make a new folder – all without leaving the editor. A rename happens in place; a move within one drive is instant; moving to the other drive copies the data across then removes the original.

Running BASIC

The editor itself contains no programming language. Japi Base BASIC is a separate product with its own manual. When the editor is built into the complete Japi Base Computer the Run menu gains a *Basic Interpreter* entry that runs the program in the active pane – but that interpreter belongs to the computer, not to the editor.

The calculator

Press **NumLock** at any time to open a built-in calculator over your text; press **NumLock** again to put it away. Your document is untouched underneath – the calculator floats on top and the editor is exactly as you left it when you close it.

There are two calculators and **Shift+Tab** switches between them:

- a **programmer's calculator** that shows one value at once in hex, decimal, signed decimal and binary, at a selectable bit width, with bitwise operations and shifts alongside ordinary arithmetic;
- a **classic calculator** for everyday floating-point sums.

Moving a number to and from your text

The calculator shares the editor's clipboard, so a result can go straight into your program:

- **Ctrl+C** in the calculator copies its current value to the clipboard. Close the calculator with **NumLock** and **Ctrl+V** drops the number into your text.
- **Ctrl+V** in the calculator pastes a number from the clipboard into the calculator, so you can pull a value out of your code and work on it.

Each calculator keeps its state while the editor is running, so a half-finished calculation is still there the next time you press **NumLock**.